

Análisis y diseño inicial del sistema de préstamos de bienes de la EPCC

Requisitos, modelo de dominio y modelo de dominio extendido (arquitectura en capas)

1. Introducción

Este informe reúne el análisis inicial de un sistema de gestión de préstamos de bienes de la Escuela Profesional de Ciencia de la Computación. Parte de un modelo conceptual, un esquema PostgreSQL y documentación técnica existente, usados como fuentes para comprender el dominio y ordenar las necesidades del proyecto.

Se presentan los tres entregables del primer examen: **(1) requisitos del sistema**, **(2) modelo de dominio** y **(3) modelo de dominio extendido** hacia una arquitectura en capas. El documento distingue entre el alcance previsto y lo realmente implementado, sin presentar como terminado lo que aún requiere implementación o validación con la EPCC.

2. Propósito y alcance

El sistema administra bienes, reservas, préstamos, renovaciones y devoluciones para la comunidad académica, ordenando el ciclo de uso de cada bien: desde la identificación del solicitante y la reserva de una unidad hasta la entrega, la devolución y el registro de su condición.

Estado: existen documentación inicial, una API mínima de salud y PostgreSQL local. Faltan los flujos completos del negocio y la persistencia funcional de las operaciones principales.

3. Requisitos del sistema

3.1 Requisitos funcionales

Tabla 1. Requisitos funcionales

ID	Requisito	Observación
RF-01	Registrar personas y cuentas de acceso.	Identifica usuarios y asocia sus credenciales o cuentas.
RF-02	Organizar catálogo, categorías, tipos, fichas y unidades físicas.	Estructura el inventario desde la clasificación general hasta cada bien físico.
RF-03	Gestionar reservas indicando solicitante, periodo y uso previsto.	Expresa la intención de uso antes de formalizar un préstamo.
RF-04	Registrar préstamos, responsables, unidad, plazo y reserva vinculada si corresponde.	Relaciona la operación con el solicitante, el bien y sus condiciones.
RF-05	Registrar entrega, devolución y condición del bien.	Deja constancia del estado del bien al inicio y al final.
RF-06	Registrar renovaciones autorizadas.	Extiende un préstamo según las reglas de la institución.
RF-07	Consultar disponibilidad, historial y operaciones.	Facilita la gestión diaria y la revisión de antecedentes.
RF-	Mantener trazabilidad de operaciones relevantes.	Permite saber qué ocurrió, cuándo y quién

08	intervino.
----	------------

Algunos requisitos se desprenden del esquema existente; otros requieren validar reglas con la EPCC: autorización, entrega, renovación, devolución y control de disponibilidad.

3.2 Requisitos de calidad

Tabla 2. Requisitos de calidad

Atributo	Descripción
Integridad de datos	Relaciones, estados y restricciones impiden registros incompletos o contradictorios y conservan referencias válidas.
Consistencia concurrente	Dos operaciones simultáneas no deben entregar una misma unidad física a personas incompatibles ni dejarla con estados contradictorios; la disponibilidad se comprueba dentro de operaciones que protejan la consistencia.
Seguridad	Cuentas, permisos y operaciones se limitan según el rol (solicitantes, administradores y otros perfiles autorizados).
Mantenibilidad	Código, esquema y reglas se organizan para modificarse sin afectar innecesariamente el resto; separación clara de responsabilidades.
Trazabilidad	Las operaciones relevantes dejan evidencia para reconstruir su secuencia y responsables (cambios de estado y participación de personas autorizadas).

Nota de alcance: aún no se han definido métricas concretas de rendimiento, disponibilidad, retención o accesibilidad.

4. Modelo de dominio

El modelo conceptual completo contempla identidad y acceso, vinculación académica, catálogo e inventario, políticas, reservas, préstamos, garantías, incidencias, sanciones y auditoría. El modelo actualmente implementable desde el SQL representa solo una parte: cuentas, inventario, reservas, préstamos y renovaciones. El siguiente diagrama de clases UML modela esa parte.

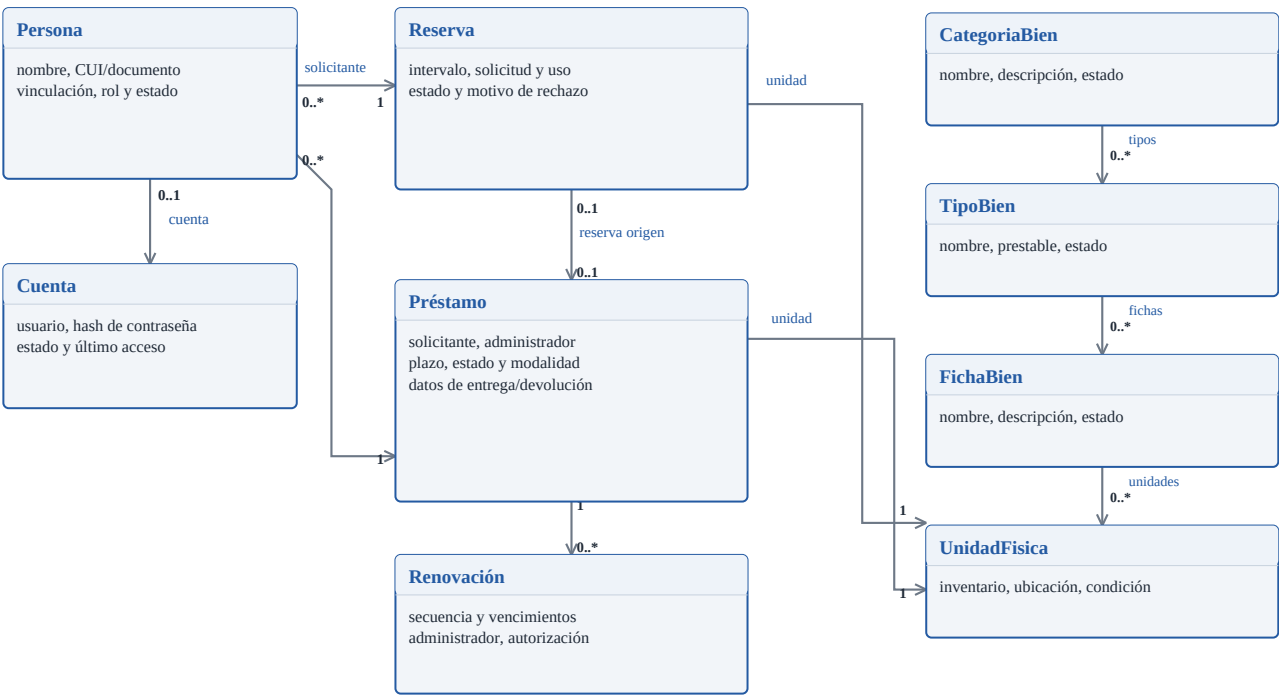


Figura 1. Diagrama de clases UML del modelo de dominio (Persona-Cuenta 1:1; jerarquía de inventario; reservas, préstamos y

renovaciones). Rediseña la tabla original de relaciones como diagrama.

Una persona tiene una cuenta; un tipo agrupa fichas; una ficha tiene varias unidades físicas. Reservas y préstamos registran quién solicita, qué unidad se usa y cómo se relacionan; una persona puede ser *solicitante* o *administrador/responsable* de un préstamo.

4.1 Hallazgos del esquema

Tabla 3. Hallazgos e implicaciones (no son errores definitivos; algunos requieren decisión de negocio)

Hallazgo	Implicación
El índice de préstamos por unidad no es único	Puede permitir inconsistencias si no se controla el préstamo activo.
El préstamo puede crearse como activo antes de registrar entrega	Separar autorización y entrega, o activar el préstamo al entregar.
Reserva y préstamo contienen referencias mutuas	Definir una relación principal para evitar redundancias.
Faltan tablas de políticas y vínculo académico	Aún no se aplican límites ni se comprueba elegibilidad.
Roles administrativos y auditoría no están desarrollados	Definir permisos y eventos antes del uso real.

5. Modelo de dominio extendido: arquitectura en capas

Se propone un **monolito modular en Python**: módulos separados por responsabilidad dentro de una misma aplicación, sin distribuir prematuramente en servicios. Facilita transacciones consistentes y reduce la complejidad inicial. Las dependencias apuntan hacia el dominio; la infraestructura se conecta a él mediante interfaces o contratos.

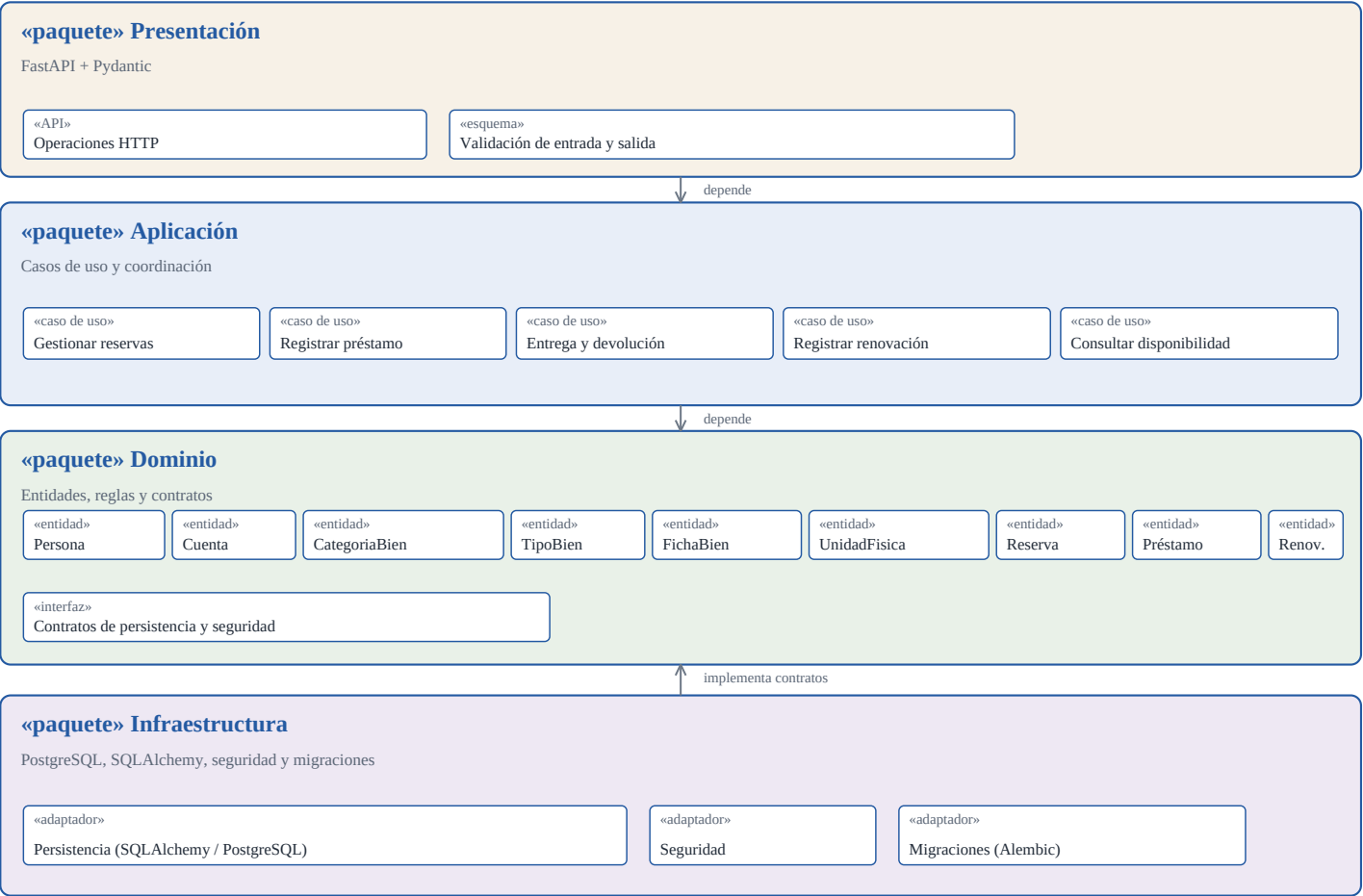


Figura 2. Diagrama de paquetes y clases UML por capas. Las entidades de Dominio son las de la Figura 1 y conservan sus

Los nombres de los casos de uso se derivan de los requisitos RF-03 a RF-07; los contratos se representan de forma genérica hasta definir las reglas con la EPCC.

6. Tecnologías propuestas

Tabla 4. Tecnologías y función

Área	Tecnologías	Función
API y validación	Python, FastAPI, Pydantic	Exponer operaciones HTTP y validar datos de entrada y salida.
Dominio y aplicación	Python tipado	Implementar entidades, reglas y casos de uso.
Persistencia	PostgreSQL, SQLAlchemy 2, Psycopg 3	Almacenar y consultar información mediante una capa de acceso estructurada.
Evolución del esquema	Alembic	Versionar y aplicar cambios controlados en la base de datos.
Calidad	pytest, Ruff, mypy	Probar, revisar y comprobar la calidad del código.
Desarrollo local	Docker Compose	Ejecutar localmente los servicios necesarios.

7. Estado actual y trabajo pendiente

Existe documentación inicial, una API mínima de salud y PostgreSQL local. Aún no están implementados los modelos de persistencia de negocio, la autenticación ni los principales casos de uso; la base actual valida la dirección del proyecto, pero no es un sistema operativo completo.

1. Integrar SQLAlchemy y Alembic.
2. Definir reglas de préstamos activos.
3. Formalizar políticas.
4. Completar identidad e inventario.
5. Validar disponibilidad de manera transaccional.

Este orden reduce riesgos: primero una base controlada de persistencia y migraciones, luego las reglas que afectan la disponibilidad y, al final, identidad, inventario y validación transaccional, con revisión en cada etapa.